

Q11. Differentiate between Bias and Variance with respect to the learning process. How does each affect model generalization? (5)

- **Bias:** Refers to the error introduced by approximating a real-world problem with a simplified model. High bias indicates the model is too simplistic, leading to **underfitting**. This results in poor performance both on the training and test datasets. Example: Linear regression applied to a non-linear problem. (1)
- **Variance:** Refers to the model's sensitivity to small fluctuations in the training data. High variance indicates the model is too complex and fits the training data too closely, leading to **overfitting**. This results in high performance on the training set but poor performance on the test set. Example: A decision tree with too many branches. (1)
- **Impact on Generalization:** A model needs to balance both bias and variance to achieve optimal generalization. High bias and low variance can lead to underfitting, while low bias and high variance can lead to overfitting. The goal is to minimize both bias and variance to improve generalization. (3)

Q12. Explain the significance of architecture design in feedforward networks. How does depth influence representational power? (5)

- **Architecture Design:** In feedforward networks, the design determines how information flows through the layers and how well the network can capture complex patterns in data. The network consists of an input layer, one or more hidden layers, and an output layer. (2.5)
- **Depth and Representational Power:** The depth of a feedforward network refers to the number of hidden layers it contains. Increasing depth increases the network's ability to capture more complex patterns, as each additional layer can learn hierarchical features. Deeper networks can represent more intricate decision boundaries, improving the network's expressiveness. However, deeper networks may also require more data to train effectively and can be prone to issues like vanishing gradients. (2.5)

Q13. Analyze the working of the backpropagation algorithm by breaking it into forward pass, error computation, and weight update steps. How do these components interact to enable efficient training of deep networks? (5)

- **Forward Pass:** The input data is passed through the network layer by layer, where each layer applies its weights and activation functions to compute outputs. The final output is used for comparison with the target labels. (1)
- **Error Computation:** The error (or loss) is calculated by comparing the network's output with the actual target values using a loss function (e.g., mean squared error for regression

or cross-entropy for classification). This represents the network's performance.(1)

- **Weight Update:** The error is then propagated backward through the network (backpropagation), and the gradients of the error with respect to the weights are computed using the chain rule. These gradients are used to adjust the weights in the opposite direction of the gradient, minimizing the error. This process is typically done using an optimization algorithm like gradient descent.(1)
- **Interaction:** The forward pass computes the network's output, the error computation determines the difference between predicted and actual outputs, and the weight update adjusts the model parameters to reduce this error. These steps enable efficient training by iteratively improving the model's performance.(2)

Q14. Explain how dataset augmentation improves model robustness. Give an example in image classification. (5)

- **Dataset Augmentation:** Dataset augmentation involves artificially increasing the size of the training dataset by applying various transformations to the original data. This improves model robustness by exposing the model to a wider variety of data, helping it generalize better and reducing overfitting.(2.5)
- **Example in Image Classification:** In image classification, augmentation techniques like rotation, flipping, zooming, shifting, and changing brightness or contrast can be applied to the images. This helps the model learn invariant features that are not dependent on specific orientations or lighting conditions. For example, flipping an image of a dog helps the model recognize the dog in different orientations, improving its robustness to real-world variations.(2.5)